通用遥操作DM-EXton2机器人适配与开发指南 2026210 (V1.1)

适用对象：机器人集成工程师、算法开发人员、科研人员

文档密级：公开

最后更新：2026-02-10

---

1. 概述 (Overview)

1.1 系统简介

本软件开发包（SDK）主要有以下几个板块：机器人反馈关节角转换DM-EXton2设备接口和Pink IK仿真模块。通过集成 **Pink** 逆运动学求解器与 **Pinocchio** 动力学库，本系统能够将DM-EXton2发送的末端位姿（Cartesian Poses）实时转换为机器人的关节指令（Joint Commands），为客户适配提供示例。本案例适配了两个机器人，分别是逐际动力Oli EDU和宇树G1-EDU。

1.2 重要风险声明

⚠【重要警告】基于Pink仿真适配计算出的/target_robot/joint_commands数据，对于使用此数据进行真实机器人控制的风险，我们不做任何保证。用户必须充分理解以下风险并采取适当的防护措施。

在使用本系统进行真实机器人控制之前，请您务必仔细阅读并充分理解以下风险声明：

1.2.1 Pink仿真模块的局限性

本软件开发包中包含的Pink IK（逆运动学）仿真模块，是基于Pinocchio库和Pink框架开发的仿真演示程序。该模块的主要目的是为用户提供一个快速验证和仿真测试的环境，而非用于直接控制真实机器人。Pink模块计算出的关节指令/target_robot/joint_commands存在以下局限性：

1. 仿真与现实的差异：仿真环境使用理想化的物理模型，不考虑真实机器人的动力学特性、摩擦、间隙、柔性等因素。仿真中可行的运动轨迹在真实机器人上可能无法实现或产生意外行为。

2. 安全限制缺失：Pink模块虽然包含基本的配置限制和速度限制，但这些限制是基于URDF模型中的理论值。真实机器人可能有更严格的安全限制要求，如力矩限制、加速度限制、工作空间限制等。

3. 未经验证的算法：Pink使用的基于二次规划的IK求解方法在数学上是严谨的，但并未针对您的具体机器人型号进行充分测试和验证。不同的机器人构型、连杆参数、关节配置都会影响IK求解的有效性。

4. 实时性不保证：Pink模块运行在ROS2的节点中，受操作系统调度、网络延迟、计算负载等因素影响。在真实机器人控制场景中，控制周期的抖动可能导致不稳定行为。

1.2.2 推荐的使用方式

我们强烈建议用户按照以下方式使用本系统：

方式一：使用自定义IK算法（推荐）

如果您拥有自己的IK逆解算法或者机器人厂家有相关接口接受末端位姿进行控制，在进行完外骨骼的接口的输入输出测试后，可以直接订阅外骨骼发布的末端目标位姿话题（/target_robot/left_ee/target和/target_robot/right_ee/target），将其作为输入，通过您的IK算法计算关节指令，直接发送给您的机器人控制器。这种方式绕过了Pink仿真模块，是最直接、最可靠的集成方式。

方式二：基于Pink进行深度定制

如果您希望基于Pink进行开发，需要对Pink模块进行深度定制，包括：根据您的机器人URDF模型重新配置IK参数、添加针对您机器人的安全限制、实现更完善的错误处理和异常恢复机制、进行充分的测试验证。即使如此，我们仍然建议您在真实机器人部署前进行专业的安全评估。

方式三：仅使用仿真功能

Pink模块作为我们提供给客户的仿真Demo，其主要价值在于：帮助用户快速理解系统工作原理、提供一个可运行的参考实现、支持在没有真实机器人的情况下进行功能演示和开发测试。在仿真环境中，您可以安全地验证遥操作逻辑、调试通信接口、开发上层应用。

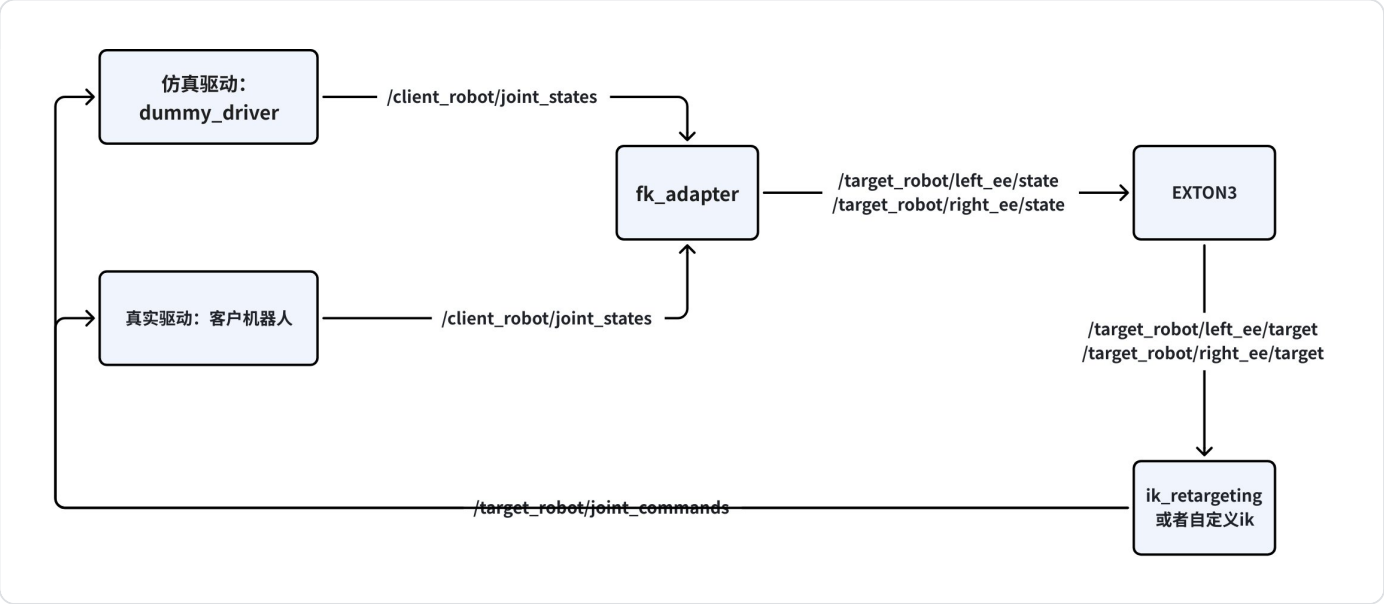
---

2. 系统架构 (System Architecture)

系统由三个核心节点组成，通过 ROS 2 话题与 TF 变换树进行通信。

2.1 数据流图

1. **反馈链路 (Robot -> Exoskeleton)**:
   - `DummyRobotDriver` (或真实驱动) 发布关节状态 `/client_robot/joint_states`。
   - `robot_state_publisher` 将关节状态转换为 TF 树。
   - `JointToPoseAdapter` 监听 TF，计算末端位姿并发布至 `/target_robot/{side}_ee/state`。



2. **控制链路 (Exoskeleton -> Robot)**:
   - DM-EXton2发布目标位姿至 `/target_robot/{side}_ee/target`。
   - `/ik_retargeting` 节点或者自定义ik 接收目标，结合当前状态解算关节位置。
   - 解算结果发布至 `/target_robot/joint_commands`。

2.2 节点清单

节点名称	对应文件	功能描述
ik_retargeting	ik_node.py	核心 IK 解算器，支持相对控制与拟人姿态。
fk_adapter	joint_to_pose_adapter.py	正运动学适配器，将关节角转为末端位姿。
dummy_driver	dummy_robot_driver.py	虚拟驱动，用于在无硬件环境下进行仿真测试。

2.3 核心话题说明

系统定义了以下核心ROS2话题，理解这些话题的用途是进行系统适配的基础：

话题名称	消息类型	频率/Hz	说明
/client_robot/joint_states	JointState	——	机器人当前关节状态（输出）
/target_robot/joint_commands	JointState	250	目标关节指令（输入）
/target_robot/left_ee/target	PoseStamped	500	左末端目标位姿（输出）
/target_robot/right_ee/target	PoseStamped	500	右末端目标位姿（输出）
/target_robot/left_ee/state	PoseStamped	——	左末端当前状态（输入）

/target_robot/right_ee/state	PoseStamped	——	右末端当前状态（输入）
/clutch/left	Bool	1000	左手Clutch使能信号
/clutch/right	Bool	1000	右手Clutch使能信号

在后续章节中，我们将详细介绍每个组件的工作原理、接口规范以及适配方法。请读者务必仔细阅读，确保对每个环节都有充分的理解。

---

3. 快速上手 (Quick Start)

3.1 环境依赖

- **操作系统**: Ubuntu 22.04 (推荐)
- **ROS 版本**: ROS 2 Humble/Iron
- **核心库**: `pinocchio`, `pink`, `numpy`, `tf2_ros`

3.2 安装系统级依赖

首先，更新软件源并安装 Pinocchio 及相关 ROS 插件依赖：

安装系统级依赖

```
sudo apt update
sudo apt install ros-humble-pinocchio
sudo apt install ros-humble-eigen3-cmake-module
```

3.3 配置 Python 虚拟环境

为了避免依赖冲突，建议在虚拟环境中安装pink及所需依赖。

3.3.1 安装 venv 模块 (如果系统中未安装)

安装 venv 模块

```
sudo apt install python3-venv
```

3.3.2 创建虚拟环境

在项目根目录下创建一个名为 `pink` （或其他自定义名称）的虚拟环境：

```
创建虚拟环境
cd exton_ros2_retargeting_demo
python3 -m venv pink
```

注意检查创建的虚拟环境是否为Python3.10的。

3.3.3 激活虚拟环境

ik逆解需要pink依赖，每次进行ik逆解之前都需要激活虚拟环境

```
激活虚拟环境
cd exton_ros2_retargeting_demo
source pink/bin/activate
```

> **注意**：激活成功后，命令行提示符前会出现 `(pink)` 标识。

3.4 安装项目依赖

在激活的虚拟环境中，使用 `pip` 安装 Python 依赖包：

```
安装项目依赖
cd exton_ros2_retargeting_demo
source pink/bin/activate
# requirements.txt在发货U盘的exton_ros2_retargeting_demo文件中
pip install -r requirements.txt
```

🚀 加速安装提示：
如果下载速度较慢，可以使用清华大学镜像源进行加速：

```
代码块
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

3.5 编译

确保代码位于工作空间 `src` 目录下，执行：

```
Bash
colcon build
source install/setup.bash
```

3.6 运行仿真 Demo

启动包含 FK 适配器、虚拟驱动和 RViz 的基础环境：

```bash
# 逐际动力Oli EDU
ros2 launch robot_fk_adapter robot_fk_adapter.launch.py use_sim:=true
# 宇树G1-EDU
ros2 launch robot_fk_adapter robot_g1_fk_adapter.launch.py use_sim:=true
```

注：此 Launch 文件会加载 `HU_D04_description` 的 URDF。

启动 IK 解算核心：

```bash
# 逐际动力Oli EDU
ros2 launch robot_ik_retargeting ik_retargeting.launch.py
# 宇树G1-EDU
ros2 launch robot_ik_retargeting g1_ik_retargeting.launch.py
```

---

4. 机器人适配指南 (Adaptation Guide)

这是本手册的核心部分。若要适配新的机器人（例如您的自定义机械臂或人形机器人），请严格按照以下步骤操作。

4.1 第一步：准备 URDF

系统完全依赖 URDF（统一机器人描述格式）进行运动学解算。

1. 确保您的机器人描述包（Description Package）和本SDK放在同一个工作空间下，并且已编译。
2. 确认 URDF 中包含准确的运动学链和关节限位。
3. **关键检查**：确认 URDF 的根节点名称（通常为 `base_link`）和您希望作为相对控制基准的节点名称（如 `waist_pitch_link`）。

4.2 第二步：配置 FK 适配器

修改 `robot_fk_adapter.launch.py` 中的参数，以匹配您的机器人 TF 树。

- **base_frame**: 设置为机器人的基座坐标系（如 `base_link`）。FK 将基于此计算末端坐标，注意正确输入ik逆解的参考坐标系，如果您的自定义ik算法基于基坐标系逆解，则请输入机器人的基坐标系（如 `base_link`），如果您的自定义ik算法基于躯干的腰部的坐标系逆解，则请输入机器人的腰部的坐标系（如 `waist_pitch_link`）。
- **注意：如果需要使用Pink逆解算法，只能使用机器人的基坐标系（如 `base_link`）**
- **left_ee_frame / right_ee_frame**: 设置为机器人左右手末端的 Link 名称（如 `left_hand_link`）。

```python
# 示例：修改 robot_fk_adapter.launch.py
Node(
    package='robot_fk_adapter',
    executable='joint_to_pose_adapter',
    parameters=[{
        'base_frame': 'base_link',              # 修改这里
        'left_ee_frame': 'link_l_hand',         # 修改这里
        'right_ee_frame': 'link_r_hand'         # 修改这里
    }]
)
```

- **如果驱动真实机器人，需要将 `robot_fk_adapter.launch.py` 的 `use_sim` 修改为 `false`，否则仿真驱动节点 `dummy_driver` 会和真实机器人冲突**

```
return LaunchDescription([
        DeclareLaunchArgument('use_sim', default_value='false',
description='Set to false for real robot'),修改这里default_value='false'
        DeclareLaunchArgument('use_rviz', default_value='true'),
```

4.3 第三步：配置 IK 解算核心（如果使用自定义ik算法可以跳过）

这是最关键的步骤，决定了解算效果和姿态自然度。请编辑 `ik_retargeting.launch.py` 文件。

4.3.1 基础链路配置

设置 URDF 路径和关键 Link 名称：

```python
# 1. 设置描述包名和 URDF 相对路径
```

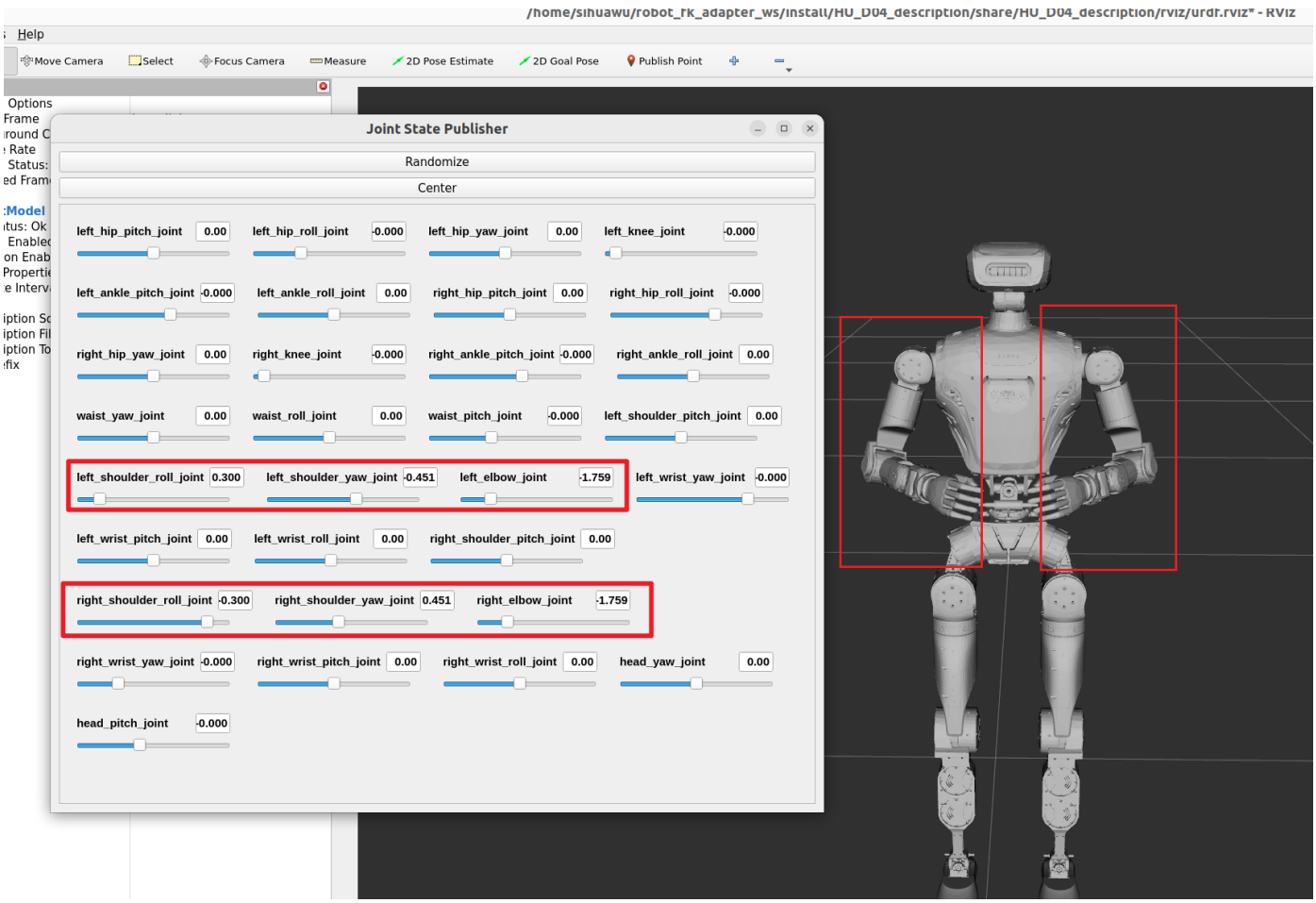
```
    pkg_name_arg = DeclareLaunchArgument('description_package',
    default_value='my_robot_description')
    urdf_file_arg = DeclareLaunchArgument('urdf_relative_path',
    default_value='urdf/my_robot.urdf')

    # 2. 设置控制目标 Link（必须与 URDF 一致）
    default_target_links = ["link_r_hand", "link_l_hand"]
```

4.3.2 拟人化姿态调优 (Human-like Config)

为了使得机器人的姿态更拟人，必须配置 `preferred_joints` 和 `preferred_angles`。

- **原理**：系统会在零空间（Null-space）中施加一个微弱的力，将关节拉向设定的期望角度。
- **调试方法**：
  a. 找到机器人肩部（Shoulder）和肘部（Elbow）的关节名称。
  b. 打开机器人Rviz仿真，拖动相关的关节，使机器人运动到肘部拟人化的姿态。



c. 确定期望角度（弧度制）。例如，肘部自然弯曲通常是 90度（±1.57 rad）。
d. **注意正负号**：不同机器人的电机安装方向不同。若肘部向后折，请反转符号。

配置示例：

```
Python
```

```
# 在 launch 文件中修改
preferred_joints_default = [
    "left_shoulder_roll", "left_shoulder_yaw", "left_elbow",  # 左臂关节
    "right_shoulder_roll", "right_shoulder_yaw", "right_elbow" # 右臂关节
]

# 对应的期望角度
preferred_angles_default = [
    0.3,   # 左肩向外张开
    0.0,   # 左肩旋转回正
    -1.5,  # 左肘弯曲 90度
    -0.3,  # 右肩对称张开
    0.0,
    -1.5   # 右肘弯曲
]
```

警告：`preferred_joints` 和 `preferred_angles` 的列表长度必须严格一致，否则节点会报错。

---

5. 基于Pink的ik_retargeting功能详解 (Detailed Features)

使用自定义ik算法控制时可参考，遥操作手柄的使能按键（Clutch）没有按下时，遥操作不会发生机器人的末端目标位姿，按下之后才会发送目标位姿：

5.1 软启动逻辑 (Soft Start)

当遥操作手柄的使能键（Clutch）按下时，系统不会立即跳变到目标位置，而是触发软启动流程。

- **触发条件**：`/clutch/{side}` 话题收到 `True` 信号的上升沿。
- **执行过程**：
  a. 记录当前物理位置为锚点（Anchor）。
  b. 重置速度缓存，防止微分冲击 。
  c. 在 `0.5` 秒内，通过插值算法将目标从锚点平滑过渡到EXton2指令。
  d. 在此期间，阻尼（Damping）权重临时提升至 `50.0`，以抑制震荡。
  e. 过渡结束后，恢复正常追踪模式，阻尼降回 `20.0`。

5.2 状态机行为

系统在 `ik_node` 中维护了一个简单的状态机：

状态	触发条件	行为描述	输出目标
锁定 (Locked)	Clutch = False	机器人保持当前姿态，不响应EXton2动作。	当前物理位姿 (T_current_phys)
软启动 (Soft Start)	Clutch Rising Edge	0.5秒内平滑过渡，大阻尼。	Interpolate(Anchor, Target, alpha)
跟随 (Tracking)	Clutch = True (t > 0.5s)	实时响应EXton2目标，包含拟人优化。	EXton2目标 (T_topic)

5.3 内部算法参数调优

部分高级参数位于代码内部，需直接修改 `ik_node.py` 文件以适应不同手感需求。

A. 滤波参数 (One Euro Filter)

位置： `init` 函数中 `self.filters` 定义处

- `mincutoff` **(最小截止频率)**: 控制**静止时的稳定性**。
  - **调大**: 响应更快，但手不动时可能会抖。
  - **调小**: 极度稳定，但会有“拖拽感”延迟。
- `beta` **(速度系数)**: 控制**快速运动时的响应**。
  - **调大**: 动态响应变快，但可能过冲。
  - **调小**: 动态响应变慢，像在水里运动。

B. IK 任务权重 (Task Weights)

位置： `init` 函数中 `self.tasks` 定义处

- `ee` **(末端任务)**:
  - `position_cost=300.0` : 位置追踪的优先级。
  - `orientation_cost=30.0` : 姿态（旋转）追踪的优先级。**调小此值可让手腕更灵活，减少手腕“死锁”现象。**
- `posture` **(姿态任务)**:
  - `cost=1.0` : **关键参数**。
  - **调大**: 机器人会强行保持“自然姿态”，可能导致手够不着目标。
  - **调小**: 机器人手能伸得更远，但肘部姿态优化可能不到位。
- `damping` **(阻尼任务)**:
  - `cost=20.0` : 防止奇异点（Singularity）飞车。一般不建议修改。

5.4 TF 可视化

为了便于调试，IK 节点会实时广播名为 `ik_target_left` 和 `ik_target_right` 的 TF 变换。

- **Parent Frame**: `root_link_name` (如 `base_link`)。
- **用途**: 在 RViz 中，您可以直接看到 IK 解算器当前正在“追逐”的那个虚幻目标点。如果机器人没动但 TF 动了，说明 IK 解算参数过硬或动力学限制生效；如果 TF 乱飞，说明EXton2数据异常。

---

6. 接口定义 (API Reference)

6.1 订阅话题 (Subscribed Topics)

- `/client_robot/joint_states` (`sensor_msgs/JointState`): 机器人当前的真实关节角度。
- `/target_robot/left_ee/target` (`geometry_msgs/PoseStamped`): EXton2发送的左手目标位姿。
- `/target_robot/right_ee/target` (`geometry_msgs/PoseStamped`): EXton2发送的右手目标位姿。
- `/clutch/left` (`std_msgs/Bool`): 左手使能信号。
- `/clutch/right` (`std_msgs/Bool`): 右手使能信号。

6.2 发布话题 (Published Topics)

- `/target_robot/joint_commands` (`sensor_msgs/JointState`): 解算后的目标关节角度，直接发给底层驱动。
- `/target_robot/{side}_ee/state` (`geometry_msgs/PoseStamped`): FK 适配器计算出的当前末端位姿。
- `/tf`: 广播 `ik_target_{side}` 用于可视化。

---

7. 故障排除 (Troubleshooting)

7.1 启动报错：`Mismatch: preferred_joints vs preferred_angles`

原因：Launch 文件中配置的拟人化关节名称数量与角度数量不一致。

解决：检查 `ik_retargeting.launch.py`，确保两个列表长度完全相等。

7.2 启动报错：`Parameter type error` (BYTE_ARRAY)

原因：ROS 2 参数传递空列表时的类型推断 BUG。

解决：代码中已通过 `ParameterDescriptor` 修复此问题。请确保 Launch 文件中不要传递空列表 `[]`，如果不需要拟人化配置，请不要在 launch 文件中声明这两个参数，让其保持默认的 `None`。

7.3 机器人手臂姿态怪异（如反关节）

原因：`preferred_angles` 设置的角度方向与电机实际正方向相反。

解决：在 RViz 中观察关节轴向，修改 `ik_retargeting.launch.py` 中的角度正负号。例如将 `-1.5` 改为 `1.5`。

7.4 上使能瞬间机器人剧烈抖动

原因：EXton2坐标系与机器人坐标系未对齐，或初始误差过大，超出了软启动的处理能力。

解决：

1. 检查 `joint_to_pose_adapter` 是否正确发布了 State。
2. 确保EXton2端使用了这个 State 作为初始偏移量的计算基准。
3. 增加代码中的 `soft_start_duration`（默认 0.5s）。

---

8. 附录：核心逻辑代码片段

8.1 拟人化姿态构建

系统会根据传入的参数动态构建 `q_natural` 向量：

```python
# 遍历参数列表，覆盖默认的零位姿态
if self.models[side].existJointName(j_name):
    jid = self.models[side].getJointId(j_name)
    idx_q = self.models[side].joints[jid].idx_q
    self.q_natural[side][idx_q] = angle
```

8.2 IK 任务栈权重分配

Python

代码块

```
# 末端追踪任务（高权重）：严格执行位置指令
"ee": FrameTask(self.ee_frames[side], position_cost=300.0,
orientation_cost=30.0),

# 姿态偏置任务（低权重）：在不影响末端的前提下优化姿态
"posture": PostureTask(cost=1.0),
```